

# Cora — Seguridad y Buenas Prácticas

Prácticas de seguridad y modelo de permisos de 3 roles (Admin, Usuario, Auditor)

Cora se diseñó desde la base de datos hacia arriba con un principio simple: cada dato de salud le pertenece a la usuaria que lo generó, y nadie más — ni siquiera un rol con privilegios elevados — accede a él sin una razón explícita y acotada. Este documento reúne las prácticas de seguridad ya implementadas y verificadas en el código, y el modelo de permisos de 3 roles (Admin, Usuario y Auditor) que gobierna quién puede hacer qué dentro del sistema. Repositorio: <https://github.com/eduardoevz/Volcanic-2026>

## 1. Buenas prácticas de seguridad implementadas

- **Row Level Security (RLS) en las 26 tablas** de la base de datos (Supabase/Postgres), verificado directamente contra el proyecto real ( `pg_tables` , `pg_policies` ): las 26 tienen RLS activo y al menos una política propia. Cada política aplica `auth.uid() = user_id` a nivel de fila: sin una sesión válida y dueña del dato, la consulta simplemente no devuelve nada — el aislamiento ocurre en la base de datos, no depende de que el cliente "se porte bien". Auditoría detallada tabla por tabla en `docs/RLS_AUDIT.md` (cobertura de la Fase 9; las tablas agregadas después siguen el mismo patrón `own_*` verificado aquí).
- **Cero secretos hardcodedos en el cliente.** La `service_role` key de Supabase y la API key de Gemini viven exclusivamente en Edge Functions server-side; el bundle de la app que corre en el teléfono de la usuaria nunca las contiene, así que no hay forma de extraerlas descompilando el APK.
- **Sesión cifrada en el dispositivo:** `LargeSecureStore` cifra con AES-256 (clave resguardada en el Keystore/Keychain nativo vía `expo-secure-store` ) antes de que cualquier dato de sesión toque `AsyncStorage` — no existen tokens en texto plano en el almacenamiento del dispositivo.
- **PKCE** en los tres flujos de autenticación (login por correo, login con Google y restablecimiento de contraseña), cerrando la puerta a interceptación del código de autorización.
- **Consentimiento explícito, granular y revocable** antes de compartir cualquier dato con la IA: el switch "Compartir contexto con Cora IA" empieza apagado por defecto para toda cuenta nueva, y la usuaria puede apagarlo en cualquier momento desde su perfil (ver `Entregables Hackathon/Interfaz y Desarrollo.md` , paso 7 del onboarding).
- **Círculo familiar con permisos de mínimo privilegio real:** sin una invitación explícitamente aceptada, el acceso es cero — nunca implícito por cercanía o parentesco. Implementado con funciones `security_definer` que devuelven únicamente señales agregadas (por ejemplo, "hoy podría necesitar apoyo"), nunca el registro ni la nota original de la usuaria.
- **Datos públicos gateados fila por fila, no por un interruptor global:** la única tabla de lectura pública ( `specialists` ) exige `consent_to_publish = true` en cada fila individual antes de aparecer en el directorio — un especialista que no dio consentimiento simplemente no existe para ninguna consulta pública, sin excepciones.
- **Superficie de ataque de inyección SQL eliminada por diseño:** todo el acceso a datos pasa por el query builder de Supabase o por RPCs parametrizados; no hay una sola consulta construida por concatenación de strings en el código de la app.
- **Código legible, tipado y consistente:** TypeScript en modo estricto, ESLint activo en CI, un módulo autocontenido por feature ( `src/features/*` ) que facilita auditar cada superficie por separado, y convenciones de commit y de marca documentadas en `docs/CONVENCIONES.md` para que cualquier persona del equipo — o cualquier jurado — pueda entender el historial sin arqueología.
- **RLS verificado automáticamente, no solo a mano:** el workflow `rls-tests.yml` de GitHub Actions corre una suite pgTAP contra la base de datos en cada cambio, comprobando en código —no solo en documentación—

que una usuaria no puede leer ni escribir filas ajenas.

## 2. Modelo de 3 roles y permisos: Admin, Usuario y Auditor

Cora separa con claridad **quién puede administrar la plataforma, quién puede usarla y quién puede auditarla**, siguiendo el mismo principio de mínimo privilegio que ya gobierna el círculo familiar y el consentimiento de IA en el código real. El rol se resuelve en la base de datos mediante una columna `app_role` en `profiles` ( `'usuario'` | `'admin'` | `'auditor'` , con `'usuario'` como valor por defecto para toda cuenta nueva) y se consulta desde las políticas RLS a través de una función `security_definer` — el mismo patrón ya usado para resolver permisos del círculo familiar.

| ROL     | QUIÉN LO TIENE                                             | PERMISOS                                                                                                                                                                                                                                                                                                                                                                                               | QUÉ TIENE EXPLÍCITAMENTE PROHIBIDO                                                                                                                                                                                                                                                                   |
|---------|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Admin   | El equipo que opera Cora                                   | Gestiona el catálogo de contenido ( <code>educational_content</code> , <code>content_categories</code> , <code>content_sources</code> ), modera el directorio de salud ( <code>health_centers</code> , <code>specialists</code> ) y mantiene catálogos compartidos ( <code>symptom_catalog</code> , <code>avatars</code> ). Su escritura está acotada exclusivamente a tablas de catálogo y contenido. | Leer <code>daily_logs</code> , <code>cycles</code> , <code>pregnancies</code> , <code>medical_background</code> o cualquier otra tabla con <code>user_id</code> de una usuaria — esas siguen protegidas por <code>auth.uid() = user_id</code> sin ninguna excepción por rol, ni siquiera para Admin. |
| Usuario | Cualquier persona registrada en la app                     | Lectura y escritura exclusiva de sus propias filas en las 26 tablas: registros diarios, ciclos, antecedentes médicos, citas, círculo familiar, conversaciones con la IA — su cuenta es, en la práctica, su propio espacio aislado.                                                                                                                                                                     | Leer o modificar datos de cualquier otra usuaria, incluso dentro de su círculo familiar (ahí solo recibe señales agregadas vía RPC, nunca la fila real).                                                                                                                                             |
| Auditor | Equipo de seguridad/cumplimiento, en revisiones periódicas | Solo lectura, y únicamente de metadatos y agregados: políticas RLS activas ( <code>pg_policies</code> ), logs de acceso, conteos y estructura de tablas. Es el rol pensado para correr la batería completa de <code>docs/RLS_AUDIT.md</code> sin necesitar ver un solo dato real de una usuaria.                                                                                                       | Escribir cualquier tabla. Leer columnas de contenido clínico o personal — notas de síntomas, mensajes de IA, antecedentes médicos — aunque sea de forma agregada.                                                                                                                                    |

## Políticas RLS que implementan el modelo

```
-- Rol de aplicación, con 'usuario' como valor seguro por defecto
alter table profiles add column app_role text not null default 'usuario'
check (app_role in ('usuario', 'admin', 'auditor'));

-- Función security definer para resolver el rol sin recursión de RLS
create or replace function public.current_app_role()
returns text language sql security definer stable as $$
select app_role from profiles where id = auth.uid();
$$;

-- Admin: escritura acotada al catálogo de contenido, nunca a datos clínicos
create policy "admin_manage_content" on educational_content
for all using (current_app_role() = 'admin');

-- Auditor: solo lectura de políticas activas, nunca de datos de usuarias
create policy "auditor_read_policies" on pg_policies
for select using (current_app_role() = 'auditor');

-- Usuario: aislamiento por fila, vigente en las 26 tablas
create policy "own_rows_only" on daily_logs
for all using (auth.uid() = user_id);
```

### 3. Por qué ningún rol es "todopoderoso"

El principio de diseño es consistente en todo el sistema: ningún rol elevado gana acceso a datos clínicos personales por defecto. Un Admin de catálogo no necesita ver el diario de síntomas de nadie para mantener la biblioteca educativa al día, y un Auditor de seguridad necesita comprobar *que las políticas existen y funcionan*, no leer el contenido que esas políticas protegen. Separar "puede administrar la plataforma" de "puede leer salud personal" es lo que evita que un solo rol comprometido — una cuenta de Admin filtrada, por ejemplo — exponga a toda la base de usuarias. Es el mismo razonamiento que ya sostiene el rediseño de permisos del círculo familiar documentado en `docs/RLS_AUDIT.md`.